Chapitre 3 : Prise en main de kubectl

Observer et administrer les ressources Kubernetes

Pr. Abdelfattah Abassi

FST Settat – Université Hassan Ier

Année universitaire 2026–2027

Plan du chapitre

À quoi sert `kubectl` ?

Définition

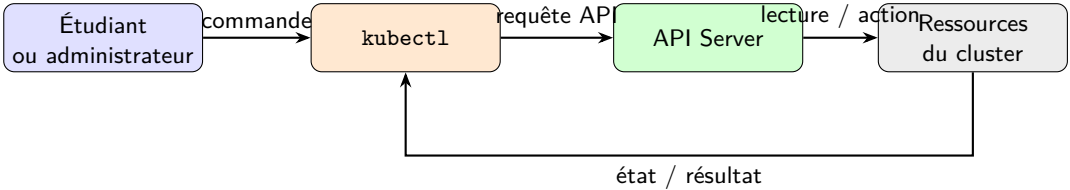
`kubectl` est l'outil en ligne de commande qui permet d'interagir avec l'API Kubernetes pour consulter et gérer les ressources d'un cluster.

- Il ne commande pas directement chaque conteneur : il envoie des requêtes à l'API Server.
- Il peut lire l'état du cluster, afficher les journaux et ouvrir un terminal dans un conteneur.
- Il peut aussi créer, modifier ou supprimer des ressources selon la commande utilisée.

Exemple d'observation

`kubectl get nodes` demande à l'API Server la liste des nuds visibles pour le contexte courant.

De la commande à la réponse



Point important

La réponse dépend du cluster et du contexte sélectionnés : un `kubectl` correctement installé peut être connecté au mauvais cluster.

Structure générale et types de commandes

Structure courante

```
kubectl VERBE TYPE [NOM] [options]
```

- **Verbe** : action demandée, par exemple `get`, `describe`, `delete`.
- **Type** : ressource visée, par exemple `pods`, `services`, `nodes`.
- **Nom / options** : ressource précise, namespace, format de sortie, etc.

Observation généralement sans modification

`get`, `describe`, `logs`, `get events`, `exec`

Modification / suppression

`apply`, `create`, `edit`, `scale`, `delete`

Nuance : exec ne modifie pas directement la définition Kubernetes, mais une commande lancée dans le conteneur peut modifier son système de fichiers ou son état applicatif.

Inspecter rapidement avec `kubectl get`

Syntaxe

```
kubectl get TYPE [NOM] [-n NAMESPACE]
```

```
kubectl get nodes
kubectl get pods
kubectl get services
kubectl get pods -o wide
```

- Sans namespace précisé, les ressources namespacées sont recherchées dans le namespace courant, souvent `default`.
- `-o wide` ajoute des colonnes utiles, notamment l'adresse IP du Pod et le nud d'exécution.
- Le résultat présente un aperçu ; il ne remplace pas une analyse détaillée.

Résultat attendu

Une liste tabulaire avec le nom, l'état, le nombre de redémarrages et l'âge des Pods (les colonnes exactes dépendent de la ressource).

Namespaces : organiser les ressources

Idée essentielle

Un namespace est un espace logique qui permet de séparer et d'organiser une partie des ressources d'un même cluster.

```
kubectl get namespaces
kubectl get pods -n kube-system
kubectl get pods --all-namespaces
kubectl get pods -n default
```

- `default` : namespace utilisé par défaut si aucun autre n'est indiqué.
- `kube-system` : contient généralement des composants système Kubernetes.
- `-all-namespaces` (ou `-A`) : affiche les ressources de tous les namespaces concernés.

Attention

Les namespaces ne sont pas des clusters séparés. Certaines ressources, comme les nuds, ne sont pas namespacées.

Labels et sélecteurs

Label

Une paire clé-valeur associée à une ressource, par exemple `app=api` ou `env=tp`.

```
kubectl get pods --show-labels
kubectl get pods -l app=api
kubectl get pods -l 'env=tp,app=api'
```

- `-show-labels` affiche les labels associés aux ressources.
- `-l app=api` sélectionne les ressources portant le label `app=api`.
- Plusieurs critères séparés par une virgule sont combinés : les ressources doivent satisfaire tous les critères.

Résultat attendu

Une liste filtrée de Pods. Si aucun Pod ne porte les labels demandés, la commande ne renvoie aucune ligne de ressource.

Obtenir les détails avec `kubectl describe`

Syntaxe

```
kubectl describe TYPE NOM [-n NAMESPACE]
```

```
kubectl describe node NOM_DU_NOEUD
kubectl describe pod NOM\_DU\_POD
kubectl describe pod NOM\_DU\_POD -n kube-system
```

- Affiche les informations détaillées : labels, état, conteneurs, conditions, volumes et événements associés.
- Permet souvent de comprendre pourquoi un Pod est en attente ou pourquoi son démarrage échoue.
- Remplacer les noms indicatifs par des noms réellement affichés par `kubectl get`.

Résultat attendu

Une sortie textuelle structurée. Lire en priorité `State`, `Conditions`, `Events` et les messages d'erreur éventuels.

Journaux et événements : comprendre ce qui se passe

Journaux : `kubectl logs`

```
kubectl logs NOM\_DU\_POD
kubectl logs NOM\_DU\_POD -c NOM_CONTENEUR
kubectl logs -f NOM\_DU\_POD
```

Affiche la sortie standard et la sortie d'erreur du conteneur. `-f` suit les nouveaux messages.

Événements : `kubectl get events`

```
kubectl get events
kubectl get events -n kube-system
kubectl get events --sort-by=.metadata.
    creationTimestamp
```

Affiche des événements comme le placement, le téléchargement d'image ou les échecs de démarrage.

Limites

Les journaux peuvent être absents si le conteneur n'a pas encore démarré. Les événements sont temporaires et leur ordre d'affichage peut dépendre de la commande utilisée.

Afficher la définition d'une ressource en YAML

Syntaxe et exemple

```
kubectl get pods NOM_DU_POD -o yaml
```

- Affiche la représentation YAML de la ressource connue de l'API.
- Permet d'inspecter les métadonnées, labels, spécification et état.
- Le résultat peut contenir des champs générés automatiquement par Kubernetes : ce n'est pas toujours un manifeste minimal à réutiliser tel quel.

Extrait illustratif :

```
metadata:
  name: exemple
  labels:
    app: demo
spec:
  containers:
  - name: web
    image: nginx
```

Exécuter une commande dans un conteneur : `kubectl exec`

Syntaxe

```
kubectl exec -it NOM_DU_POD - COMMANDE [ARGUMENTS]
```

```
kubectl exec -it NOM\_DU\_POD -- sh
kubectl exec NOM\_DU\_POD -- ls /
kubectl exec -it NOM\_DU\_POD -c NOM_CONTENEUR -- sh
```

- `-i` conserve l'entrée interactive ; `-t` alloue un terminal.
- `-` sépare les options de kubectl de la commande exécutée dans le conteneur.
- Un conteneur minimal peut ne pas contenir `bash` ou même `sh`.

Précaution

`exec` est utile pour diagnostiquer, mais toute commande exécutée peut changer l'état du conteneur. Ne pas le confondre avec une simple commande de lecture.

Déployer une définition avec `kubectl apply -f`

Principe

Un fichier YAML décrit l'état souhaité. `apply` transmet cette définition à l'API Kubernetes pour créer ou mettre à jour la ressource.

```
kubectl apply -f pod-demo.yaml
kubectl apply -f dossier-manifestes/
kubectl get -f pod-demo.yaml
```

- `-f` indique un fichier ou un répertoire de manifestes.
- `apply` est une commande de **modification** : vérifier le fichier avant de l'exécuter.
- `get -f` permet d'afficher la ressource identifiée par le fichier, si elle existe.

Résultat attendu

Un message tel que `pod/... created` ou `configured`, puis la possibilité d'inspecter la ressource avec `kubectl get`.

Supprimer une ressource avec `kubectl delete`

Syntaxes usuelles

`kubectl delete TYPE NOM` ou `kubectl delete -f fichier.yaml`

```
kubectl delete pod NOM\_DU\_POD
kubectl delete -f pod-demo.yaml
kubectl get pods
```

- La suppression modifie l'état du cluster : la ressource est supprimée via l'API.
- Le dernier `kubectl get pods` sert à vérifier le résultat.
- Si le Pod appartient à un contrôleur (par exemple un Deployment), celui-ci peut en créer un autre pour respecter l'état souhaité.

Règle de sécurité

Vérifier le type, le nom et le namespace avant de supprimer. Ne jamais copier une commande destructive sans comprendre sa portée.

TP court : observer le cluster sans déployer d'application

Objectif : produire un état des lieux argumenté du cluster installé au chapitre 2.

1. Vérifier le contexte et le cluster : `kubectl config current-context`, puis `kubectl cluster-info`.
2. Lister les nuds : `kubectl get nodes -o wide`. Noter leur état et leur version.
3. Lister les namespaces : `kubectl get namespaces`.
4. Observer les Pods système : `kubectl get pods -n kube-system -o wide`.
5. Choisir un Pod système et exécuter `kubectl describe pod NOM -n kube-system`.
6. Afficher les événements : `kubectl get events -n kube-system`.
7. Pour un Pod en cours d'exécution, consulter ses journaux avec `kubectl logs` si disponibles.

Compte rendu à remettre

Présenter les commandes exécutées, expliquer deux ressources observées, commenter un état ou un événement et préciser la différence entre une commande d'observation et une commande de modification. Ne rien supprimer ni déployer pendant ce TP.

Synthèse et vérification des acquis

- `kubectl` communique avec le cluster par l'intermédiaire de l'API Server.
- `get` liste ; `describe` détaille ; `logs` affiche les journaux ; `get events` aide au diagnostic.
- Les namespaces délimitent un espace logique ; les labels et sélecteurs filtrent les ressources.
- `exec` exécute une commande dans un conteneur : l'action lancée peut modifier son état.
- `apply -f` applique un manifeste ; `delete` supprime une ressource.

Question de contrôle

Un Pod n'apparaît pas dans `kubectl get pods`. Quelles commandes utiliseriez-vous pour vérifier le namespace, rechercher les ressources et comprendre un éventuel problème ?

Documentation : `https://kubernetes.io/docs/reference/kubectl/` et `https://kubernetes.io/docs/concepts/overview/working-with-objects/`